

Interval Asynchronous Parallel SGD - Report

Anushree C - CO23BTECH11003 , Gokul Krishna - CO23BTECH11006

November 2025

Design

Input

The project uses a CSV file as its primary input for the dataset. Each row in this file contains four numerical values, where the first three columns X_1, X_2, X_3 represent the input features and the fourth column represents the target/output value Y . This dataset was generated for a simple curve-fitting task involving a function of three input variables. The SGD process iteratively updates the model parameters so that the predicted output becomes closer to the true Y values in the dataset.

In addition to the dataset, the program accepts the number of worker nodes through the command line. Each worker operates as an independent computation unit responsible for handling a portion of the data, computing gradients, and interacting with the central dispatcher. This setup replicates a distributed training environment, allowing parallel execution of SGD steps. The number of workers supplied by the user determines how many such parallel units participate in the curve-fitting process and directly affects the overall throughput of the system.

Logic

The core idea of our approach is to distribute the Stochastic Gradient Descent process across multiple worker nodes while keeping training stable and avoiding the drawbacks of fully asynchronous updates. Instead of allowing unrestricted updates, the system divides execution into structured intervals. Within an interval, several workers can proceed independently, compute gradients on different portions of the dataset, and send their results to a central dispatcher. The dispatcher controls when a worker may begin a step and decides whether a computed gradient should be accepted based on the current interval boundaries. This controlled asynchrony allows the system to benefit from parallelism while ensuring that updates remain well-behaved and do not accumulate unbounded staleness.

Worker Functionality

Each worker process is responsible for performing the actual gradient computation. Internally, a worker first requests permission from the dispatcher to begin a new step. Once allowed, it reads the appropriate data sample or batch from the dataset, makes a temporary local copy of the current model parameters, and computes the gradient needed for the curve-fitting task. After this computation, the worker contacts the dispatcher again to check whether its gradient is still valid within the active interval. If accepted, the worker’s result contributes to updating the shared global model; if rejected, the worker simply moves on to its next step. In this way, workers continuously cycle between requesting steps, computing gradients, and coordinating with the dispatcher.

Dispatcher Functionality

The dispatcher acts as the central decision-maker of the system. It controls the progression of training by maintaining key counters that define each interval and determining which worker steps are allowed to proceed. When a worker requests to start a step, the dispatcher returns a step index and checks whether the worker is allowed to continue. When a gradient arrives, the dispatcher verifies whether the step belongs to the currently active interval and whether the interval has room to accept more updates. If both conditions are satisfied, the dispatcher applies the gradient to the global model and updates its internal counters. When an interval completes, the dispatcher advances to the next one by resetting counters and reducing the interval size as part of the staleness-control strategy.

Conclusion

Through this coordinated interaction between workers and the dispatcher, the system successfully performs distributed SGD while preventing excessive staleness. Workers operate in parallel, increasing throughput, whereas the dispatcher enforces structured boundaries to preserve training stability. As execution proceeds across intervals, the global model gradually improves and moves closer to the curve that best fits the dataset. Ultimately, the system demonstrates how interval-based asynchrony can be combined with a distributed execution model to achieve efficient and reliable optimization for a simple curve-fitting problem.

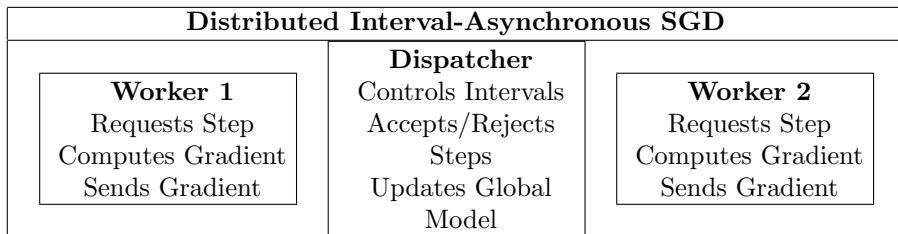


Figure 1: View of the Dispatcher and Worker interactions in Interval-Asynchronous SGD for two workers

Related work and Progress Achieved

Referred paper approach

Interval-Asynchronous SGD was introduced as a way to control gradient staleness by dividing execution into structured intervals instead of allowing completely free asynchronous updates. The original paper showed that this interval-based strategy improves stability and convergence compared to naïve asynchronous methods. However, the authors implemented their approach strictly in a shared-memory multithreaded environment, where threads operate on a common model and rely on atomic instructions like CAS and FAA. This design depends heavily on the properties of single-machine shared memory, making it unsuitable for distributed systems.

Limitations of that approach

Because their method runs only on one machine, it cannot handle key challenges that naturally appear in distributed training—such as network delay, message reordering, communication overhead, or independent worker failures. The original algorithm also assumes instantaneous visibility of shared variables, which breaks down completely when workers run as separate processes across different machines. Hence, although interval-asynchrony is a promising idea, its original formulation cannot be applied directly to distributed SGD.

Progress in Field

Distributed SGD itself has been studied extensively in the context of large-scale machine learning. Prior work has mainly focused on three families of approaches: synchronous training, which provides strong consistency at the cost of significant waiting; fully asynchronous training, which improves speed but often suffers from high staleness; and bounded-staleness or parameter-server models, which attempt to strike a balance between the two. Despite these developments, none of the existing methods incorporate interval-based control, and interval-asynchrony has never been implemented in a distributed environment. Coordinating interval boundaries across distributed workers requires rethinking

the entire dispatcher design, which is likely why this direction has remained unexplored.

Our contribution

Our work fills this gap by extending interval-asynchrony beyond its original multithreaded setting to a fully distributed execution model. We replace threads with separate worker processes and use ZMQ sockets for communication, allowing us to serialize update decisions at the dispatcher instead of relying on atomic hardware operations. This demonstrates how interval-controlled SGD can function correctly under realistic distributed conditions, handling delayed messages, independent worker execution, and non-shared memory. In this way, our project takes a concept that until now existed only in multithreaded research and implements it in a distributed setup.

Results

We generated a synthetic dataset of 500 datapoints using the function

$$y = 3X_1 + 2X_2 + 5X_3,$$

where X_1, X_2, X_3 are input features and y is the target value.

Converged Parameters

After running the distributed Interval-Asynchronous SGD implementation, the model parameters converged close to the ground truth coefficients. The final learned parameters obtained from the log file are shown below:

Parameter	Learned Value
w_1	3.001370
w_2	1.996412
w_3	4.995865
w_4	0.004415

Table 1: Learned model parameters from the distributed SGD run

These values can be compared to the true coefficients $(3, 2, 5)$, demonstrating that the distributed training system successfully converges to the correct solution. w_4 is the converged value of the constant term. in our case its 0 , as there is not constant term in $3X_1 + 2X_2 + 5X_3$

Error Plot

To evaluate convergence behaviour, we also recorded the error value (loss function) at each iteration. The following figure shows the error decreasing over time:

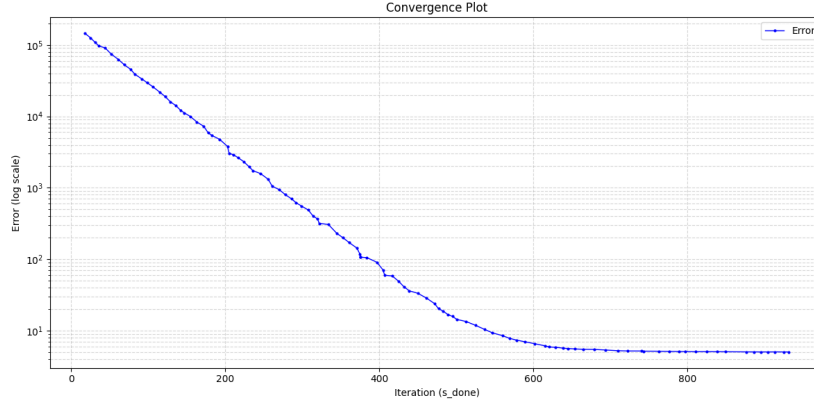


Figure 2: Error vs. Iteration plot for distributed Interval-Asynchronous SGD

The downward trend in the plot confirms that the model steadily learns from the dataset and converges towards the true underlying parameters.

Limitations

- The dispatcher is single-threaded (becomes bottleneck)
- No fault tolerance
- Only linear regression used
- No real networking test (local processes only)

Future Work

- Future extensions of this work could involve scaling the system to multi-machine clusters, where network variability and higher latency offer a more realistic distributed environment.
- Allow workers to compute gradients on mini-batches or multiple samples at once, enabling more realistic deep-learning experiments.